

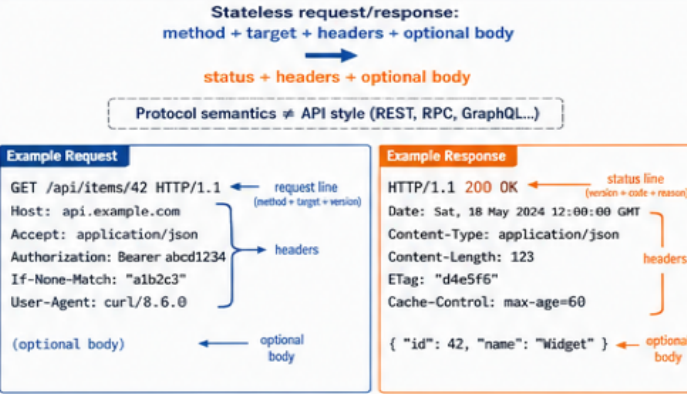
HTTP

The application protocol of the web

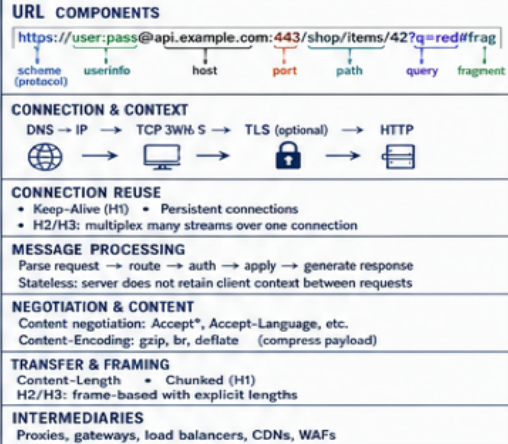
1 WHY

- Standard, interoperable client/server exchange
- Resource semantics (methods + status codes)
- Extensible metadata (headers, content types)
- Intermediaries enable proxies, gateways, CDNs
- Caching reduces latency and load

2 CORE IDEA



3 INTERNALS



4 METHODS / SEMANTICS

COMMON METHODS					Typical Use
Method	Safe	Idempotent	Cacheable		
GET	✓	✓	✓		Read resource
HEAD	✓	✓	✓		Headers only
POST	✗	✗	(✗)		Create/Process
PUT	✗	✓	(✓*)		Replace resource
PATCH	✗	(✓*)	✗		Partial update
DELETE	✓	✓	(✗)		Remove resource
OPTIONS	✓	✓	✗		Capabilities (CORS)

STATUS CODE FAMILIES

1xx Informational

2xx Success

3xx Redirection

4xx Client Error

5xx Server Error

COMMON CODES

200 OK

201 Created

204 No Content

301 Moved Permanently

302 Found

304 Not Modified

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

429 Too Many Requests

500 Internal Server Error

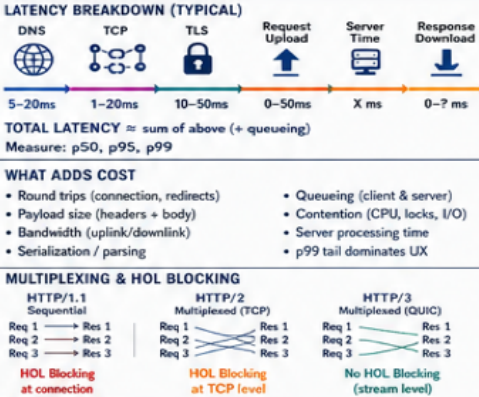
503 Service Unavailable

SEMANTICS

- Safe: no state change (GET, HEAD, OPTIONS)
- Idempotent: same result if repeated (GET, HEAD, PUT, DELETE, OPTIONS)
- Cacheable: by default or explicit rules (GET, HEAD, POST only with explicit headers)

(*) depends on headers / caching rules

5 COMPLEXITY



6 TRADE-OFFS

Benefits	Costs / Constraints
✓ Simplicity & ubiquity	✗ Header overhead (bytes)
✓ Interoperability	✗ Stateless: more data each request (auth, context)
✓ Observability	✗ Version complexity (H1/H2/H3) + tooling
✓ Caching & offload	✗ Intermediaries may transform, cache or block
✓ Extensibility	✗ Semantics misuse (e.g., POST for everything) hurts cache/UX
✓ Connection reuse & multiplexing	✗ HOL blocking (H1), TCP congestion, middleboxes
✓ Security (TLS)	✗ CPU/latency for crypto, cert management
✓ Large ecosystem	✗ Heterogeneous behavior (across clients/proxies/CDNs)

7 DISCUSSION

- Idempotency: enables safe retries; prefer PUT/PATCH/DELETE for idempotent writes
- Retries: only for safe/idempotent; use exponential backoff + jitter
- Caching: validators (ETag/If-None-Match, Last-Modified/If-Modified-Since)
- Cache-Control: max-age, s-maxage, no-cache, no-store, public, private, stale-if-error
- Cookies: state management; Secure, HttpOnly, SameSite
- Content types: negotiate via Accept; set Content-Type correctly
- Redirects: 301/308 (preserve method) vs 302/307 (client dependent)
- Range requests: Range: bytes=... → 206 Partial Content
- Streaming: chunked (H1), streaming responses; gRPC over H2/H3
- Keep-Alive: tune timeouts; idle connection reuse
- Timeouts: connect, TLS, request, response; set deadlines
- Backpressure: flow control (H2/H3), TCP windows
- Failure modes: 4xx vs 5xx vs network errors; partial failures
- Retry storms: coordinate clients; use token buckets & jitter

8 RELATED

HTTPS / TLS	Encryption, integrity, authentication
REST	Architectural style over HTTP
DNS	Name resolution
TCP / QUIC	Transport protocols
Proxies / Gateways	Routing, auth, transforms
CDN	Caching at the edge
Cookies	Client-side state
CORS	Cross-origin requests
WebSockets	Full-duplex over HTTP Upgrade

9 PRODUCTION

Browsers	Page loads, assets, prefetch, service workers
APIs	Public/private APIs, mobile & web clients
Microservices	Service-to-service communication
Object Storage	S3-compatible, backups, large objects
CDNs	Static assets, media, global acceleration
Load Balancers	Layer 7 routing, health checks
Observability	Logs, metrics, traces, dashboards, alerts
Key Metrics	RPS • Error rate (4xx/5xx) • p50/p95/p99 latency Throughput (bytes) • Cache HIT ratio • Retries

10 INTERVIEW THINKING

- Clarify: what is the resource? read or write? who are the clients?
- Trace the full request path: client → DNS → TCP/QUIC → TLS → proxy/CDN → server
- Define semantics: correct method, status codes, headers, idempotency
- Plan caching: cacheability, validators, TTL, cache keys, Vary
- Plan retries & timeouts: idempotent only, backoff, jitter, deadlines
- Quantify: list latency components; identify biggest contributors
- Identify failure modes & mitigations; choose HTTP version; observe everything

11 SENIOR NOTES

- ✓ Set explicit deadlines for every hop (connect/TLS/request/response)
- ✓ Retry only safe/idempotent operations with exponential backoff + jitter
- ✓ Bound request/response bodies; stream where possible
- ✓ Validate and sanitize untrusted metadata (headers, URLs, cookies)
- ✓ Design cache keys and settings intentionally; avoid cache poisoning
- ✓ Instrument every hop: metrics, logs, traces, correlation IDs
- ✓ Distinguish transport success from application success (2xx vs 4xx/5xx)

MENTAL HOOK

“HTTP is a semantic contract carried over connections: methods, status, caching, and failure behavior matter more than JSON.”

HTTPS

HTTP protected by TLS authentication and encryption

1 WHY



Confidentiality
Encrypts data to stop eavesdropping



Integrity
Detects tampering in transit



Server Authentication
Verifies you're talking to the right server



Optional Client Auth
Verifies the client with mTLS



Against eavesdropping, tampering, MITM attacks

4 COMPLEXITY



Extra Handshakes / RTTs
Adds delay (TCP + TLS handshake).



CPU Cryptography
Handshakes and encryption are CPU costly.



Certificate Validation
Chain building, OCSP/CRL checks.



Record Overhead
Headers + tags add bytes.



Connection Reuse
Keep-alive, HTTP/2 multiplexing, HTTP/3.



Resumption / 0-RTT
Reduces latency; adds complexity & risk.



TLS Termination
At proxies/LBs; key mgmt & trust boundary.



Latency Impact
Affects p50/p99; handshake dominates.

2 CORE IDEA

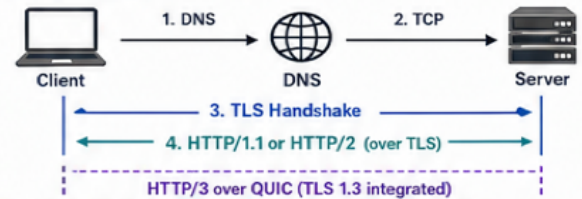


- HTTP semantics travel inside a TLS-secured channel.
- Public-key cryptography authenticates and establishes shared secrets.
- Symmetric AEAD ciphers protect application records for confidentiality and integrity.
- Encryption does not make the endpoint or application trustworthy.



3 INTERNALS

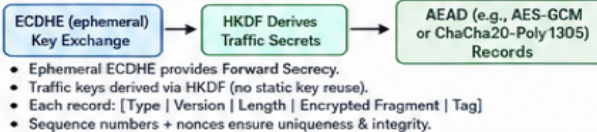
Connection Flow



TLS 1.3 Handshake (Typical)



Key & Record Internals (TLS 1.3)



Advanced Features



5 TRADE-OFFS

Strong Transport Security	VS	Cert Ops & Lifecycle Overhead
Confidentiality & Integrity	VS	CPU & Latency Cost
Authentication & Trust	VS	Metadata Leakage (SNI, cert info)
Global Interop	VS	Protocol / Cipher Complexity
End-to-End Security	VS	Termination Trust & Key Access
User Experience	VS	Debugging & Observability

DISCUSSION

- TLS vs HTTPS: TLS is the protocol; HTTPS = HTTP over TLS.
- What's encrypted: HTTP headers + body (not DNS, TCP, or SNI).
- Trust stores: OS/browser root stores define trusted CAs.
- Cert rotation: Short-lived certs reduce risk; automate with ACME.
- Wildcard certs: *.example.com risk spans all subdomains.
- Mixed content: HTTP resources break security; block/upgrade.
- Downgrade prevention: Use TLS 1.2/1.3 only; enable HSTS.
- Pinning: Avoid in browsers; use with caution (breaks recovery).
- mTLS: Strong client identity; cert mgmt is harder.
- Reverse proxies: Offload TLS; ensure backend encryption if needed.
- End-to-end vs Edge termination: Decide per trust boundary.
- 0-RTT: Improves latency; replayable—use idempotent-safe only.
- Clock skew: Can break validity; keep time (NTP) in sync.
- Failure modes: Expired cert, handshake fail, OCSP down, MITM.

RELATED

- HTTP (Semantics, Methods, Status Codes)
- TLS / PKI (X.509, CAs, OCSP, CRLs)
- Certificates: (ACME, Let's Encrypt)
- DNS (Resolution, CAA, DNSSEC)
- TCP / QUIC (Transport, Congestion Control)
- HSTS (Enforce HTTPS)
- Reverse Proxies / Load Balancers (Nginx, Envoy, ALB)
- Service Mesh (mTLS, Identity)
- Secrets Management (Vault, KMS, HSM)

8 PRODUCTION

- ✓ Browser Websites: Serve all traffic via HTTPS + HSTS.
- ✓ APIs: Require HTTPS; validate tokens, rate limit.
- ✓ Ingress / Load Balancers: Terminate TLS; use modern ciphers.
- ✓ CDNs: Perform TLS termination close to users.
- ✓ Service-to-Service: Use mTLS within private networks.
- ✓ Certificate Automation: ACME, auto-renew, short TTL.
- ✓ Monitoring: Track cert expiry, handshake errors, latency.
- ✓ Observability: Log TLS versions/ciphers, failures, SNI.

INTERVIEW THINKING

- ✓ Define threat model and trust boundary.
- ✓ Trace the connection: DNS → TCP → TLS → HTTP.
- ✓ Walk the TLS 1.3 handshake step by step.
- ✓ Explain how server identity is validated.
- ✓ Discuss what is encrypted and what is not.
- ✓ Choose termination points and justify.
- ✓ Plan certificate lifecycle and key management.
- ✓ Quantify latency/cost; consider resumption, H2/H3.
- ✓ Test failure cases and monitoring/alerts.
- ✓ Make security and operational trade-offs explicit.

SENIOR NOTES

- ✓ Automate renewal and rotation; prefer short-lived certs.
- ✓ Encrypt every hop where the boundary requires it.
- ✓ Never disable verification or accept invalid certs.
- ✓ Protect private keys (HSM/KMS); restrict access.
- ✓ Prefer TLS 1.2/1.3 and strong, forward-secret ciphers.
- ✓ Keep libraries, ciphers, and protocols current.
- ✓ Make timeouts and retries explicit; avoid slow fail.
- ✓ Treat 0-RTT as replayable; design idempotent ops.
- ✓ Monitor expiry, handshake errors, and negotiation.
- ✓ Document trust boundaries and runbooks.

11 MENTAL HOOK

HTTPS protects the journey, not the destination:
TLS secures bytes in transit while application security still decides trust.



REST

Resource-oriented architecture over uniform interfaces

1 WHY



Web-scale interoperability
Leverage HTTP, URIs, media types



Loose coupling
Independent evolution of clients and servers



Evolvable systems
Change behind stable interfaces



Cacheability
Improve performance, reduce load and latency



Intermediary support
Proxies, gateways, CDNs, filters

2 CORE IDEA

REST is an **architectural style**, not synonymous with JSON over HTTP.

Model resources with stable identifiers and manipulate representations through a uniform interface.

REST ARCHITECTURAL CONSTRAINTS (Roy Fielding)

- 1 Client-Server** Separation of concerns
- 2 Stateless** No client context on server between requests
- 3 Cacheable** Responses define themselves as cacheable
- 4 Uniform Interface** Consistent, self-descriptive interactions
- 5 Layered System** Intermediaries between client and server
- 6 Optional Code-on-Demand** Extend client functionality (e.g., JS)

3 INTERNALS



Resource URIs

- Nouns, hierarchical, stable identifiers
- /users, /users/{id}, /orders/{id}/items



Representations & Content Negotiation

- JSON, XML, CBOR, etc
- Accept, Content-Type headers



HTTP Methods & Status Semantics

- GET, POST, PUT, PATCH, DELETE, HEAD
- 2xx success, 3xx redirection, 4xx client error, 5xx server error



Headers

- Auth, caching, conditionals, tracing
- Authorization, Cache-Control, ETag



Links & Hypermedia

- Discoverability via links
- HATEOAS where practical



Conditional Requests with ETag

- If-None-Match / If-Match
- 304 Not Modified / 412 Precondition Failed



Caching

- Cache-Control, Expires, Vary
- Strong/weak validators with ETag



Authentication per Request

- No server-side session
- Tokens/creds sent on every request

REQUEST/RESPONSE FLOW



4 SEMANTICS

METHOD	SAFETY	IDEMPOTENCY	TYPICAL USE
GET	Safe	Idempotent	Read resource
HEAD	Safe	Idempotent	Metadata only
PUT	Unsafe	Idempotent	Replace resource
DELETE	Unsafe	Idempotent	Remove resource
POST	Unsafe	Not idempotent	Create / action
PATCH	Unsafe	Depends	Partial update

STATUS CODE CLASSES

2xx	3xx	4xx	5xx
Success	Redirection	Client Error	Server Error

- Collection vs Item URIs**
/orders (collection) vs /orders/{id} (item)
- Pagination**
?page=2&size=20
Link: <...>?page=3; rel="next"
- Filtering**
?status=shipped&createdAt=2024-01-01
- Sorting**
?sort=-createdAt,name
- Versioning**
/v1/orders or Accept: application/vnd.myapi.v2+json
- Optimistic Concurrency**
ETag + If-Match for updates/deletes
- Error Envelopes (example)**

```
{  "error": {    "code": "ORDER_NOT_FOUND",    "message": "Order not found",    "details": {      "orderId": "123"    }  }}
```

5 COMPLEXITY

- Network Round Trips**
 - High latency dominates
 - Minimize calls, prefer coarse resources
- Payload & Serialization**
 - JSON overhead, compression helps
 - Binary formats reduce size
- Chatty APIs**
 - Many small calls increase p50/p99
 - Combine or batch where sensible
- Over-fetching / Under-fetching**
 - Too much data wastes bandwidth
 - Too little data causes more calls
- Caching Gains**
 - Cacheable GETs reduce load/latency
 - Leverage CDN, client, proxy caches
- Pagination Cost**
 - Offset pagination can be slow
 - Prefer cursor/seek for large datasets
- Fan-out**
 - Aggregations across services increase
 - Tail latency (p99)
- Latency Metrics**
 - Optimize for p50 and p95/p99
 - Measure client-perceived latency (RUM)

6 TRADE-OFFS

STRENGTHS

- ✓ Simple mental model
- ✓ Ubiquitous tooling
- ✓ Intermediary & cache support
- ✓ Loosely coupled evolution
- ✓ Works well at web scale

LIMITATIONS

- ✗ Weaker contracts by default
- ✗ Payload verbosity/overhead
- ✗ Multiple round trips
- ✗ Can be misused (verbs in URI, ignoring semantics)

REST IN CONTEXT

PARADIGM	BEST FOR	TRADE-OFFS
REST	General-purpose web APIs, CRUD, public Internet	More round trips, payload overhead
RPC (e.g., gRPC)	Low-latency calls, strong contracts	Tighter coupling, gateway/proxy limits
GraphQL	Flexible reads, reduce over-fetching	Cache complexity, server complexity
Event-Driven	Async integration, decoupled producers	Eventual consistency, operational complexity

7 DISCUSSION

- ✓ Model resources, not operations; prefer nouns
- ✓ Partial updates with PATCH; define semantics
- ✓ Idempotency keys for POST to enable safe retries
- ✓ Retries: idempotent only; add backoff + jitter
- ✓ Long-running ops: return 202 + monitor URI
- ✓ Bulk operations: weigh efficiency vs semantics
- ✓ HATEOAS: practical in some domains, not dogma
- ✓ API evolution: additive changes, deprecation plan
- ✓ Backward compatibility: avoid breaking changes
- ✓ Rate limits: communicate via headers (429)
- ✓ Authorization: least privilege per resource/action
- ✓ Multi-tenancy: tenant in token or path; isolation

8 RELATED

- HTTP/HTTPS — transport & semantics
- OpenAPI (OAS) — contract & docs
- JSON — common representation
- gRPC — RPC over HTTP/2
- GraphQL — query flexibility
- Caching — CDN, proxies, client caches
- API Gateways — routing, auth, quotas
- OAuth 2.0 / OIDC — auth & identity

9 PRODUCTION

- Public APIs (partners, developers)
- CRUD Services (business data)
- Mobile / Web Backends
- Control Planes & Admin APIs
- Object Stores & File Services
- Microservices (service-to-service)
- API Gateways & Aggregation Layers

10 INTERVIEW THINKING

- 1 Clarify consumers & workflows
- 2 Identify resources & invariants
- 3 Define operations & semantics
- 4 Design errors, pagination, versioning
- 5 Plan retries, caching & security
- 6 Choose sync vs async patterns
- 7 Quantify chattiness & latency impact
- 8 Observe, measure, iterate

11 SENIOR NOTES

- ★ Design around workflows, not tables
- ★ Make mutation retry semantics explicit
- ★ Preserve backward compatibility
- ★ Authorize each resource/action
- ★ Bound collections & payload sizes
- ★ Expose concurrency control (ETag)
- ★ Document with executable schemas
- ★ Measure client-perceived latency

MENTAL HOOK

REST scales through constraints: stable resources, uniform semantics, stateless exchanges, and cacheable representations.

gRPC

Contract-first remote procedure calls over HTTP/2

1 WHY



Strongly typed contracts
schema as the source of truth



Code generation
stubs & interfaces for all major languages



Efficient binary messages
Protocol Buffers over HTTP/2
small, fast, compressed



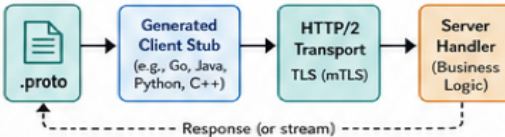
Cross-language service calls
polyglot clients & servers



Streaming & multiplexing
low latency, high throughput
many RPCs over one connection

2 CORE IDEA

- Define services and messages in Protocol Buffers (.proto)
- Generate client stubs and server interfaces
- Client calls a method as if local; network and failure semantics remain remote



example.proto

```
service Greeter {  
  rpc SayHello (HelloReq)  
    returns (HelloResp);  
}  
  
message HelloReq { string name = 1; }  
message HelloResp { string msg = 1; }
```

Generated (Go)

```
// Client stub  
func (c *greeterClient) SayHello(  
  ctx context.Context, in *HelloReq, opts ...grpc.CallOption) (*HelloResp, error) {  
  // Server interface  
  type GreeterServer interface {  
    SayHello(ctx context.Context, in *HelloReq) (*HelloResp, error)  
  }
```

4 COMPLEXITY

Area	What to reason about
Serialization	Protobuf encode/decode cost; CPU impact
Network Round Trips	latency, handshake, TLS resumption
Per-Message Framing	5B header + payload; compression trade-offs
Flow Control	HTTP/2 window updates; backpressure
Stream Concurrency	many streams / connection; head-of-line avoidance at stream level
Connection Warm-up	TCP + TLS + HTTP/2 preface; keepalive
Payload Sizes	message & max receive size limits
CPU & Memory	(de)serialization, copies, GC pressure
Latency Distribution	p50 vs p99; tail latency amplification

Binary efficiency does not remove network latency.
Measure, don't assume.

5 TRADE-OFFS

What you gain

- Strong contracts & codegen
- High performance (binary, HTTP/2, multiplexing, HPACK)
- Streaming & low latency
- Cross-language & platform
- Tooling & ecosystem
- Rich status & metadata
- Schema evolution controls

What you give up / watch

- Browser friction (no native gRPC; needs gRPC-Web / proxy)
- Debugging opacity (binary)
- Schema discipline required
- Proxy / firewall compatibility
- Tighter coupling via contracts
- Operational complexity (channels, LB, retries)

gRPC vs Alternatives

Aspect	gRPC	REST (HTTP/1/2)	Async Messaging (Kafka, NATS, ...)
Interaction	Sync request/response & streaming	Sync request/response (resources)	Async publish/subscribe (event driven)
Contract	.proto (strongly typed)	OpenAPI/JSON	Schema (Avro/Protobuf) or schemaless
Transport	HTTP/2 (binary)	HTTP/1.1 or HTTP/2 (JSON/text)	Persistent log / queues
Best for	Service-to-service calls, low latency, streams	Public APIs, browsers, simplicity	Decoupling, fan-out, event processing
Trade-offs	Not browser-native, steeper learning	Higher payload, less efficient	Eventual consistency, operational overhead

7 RELATED

- HTTP2** HTTP/2 – multiplexing, streams, HPACK, flow control
- Protocol Buffers** – schema, binary encoding, evolution
- RPC** RPC (Remote Procedure Call) – abstraction for invoking remote functions
- REST** REST – resource-oriented HTTP APIs (stateless, cacheable)
- TLS / mTLS** – transport security & identity
- Service Mesh** – traffic mgmt, mTLS, retries, observability
- Load Balancing** – client/server LB, discovery, outlier detection
- Distributed Tracing** – context propagation, spans, baggage
- Observability** – metrics, logs, traces, SLOs

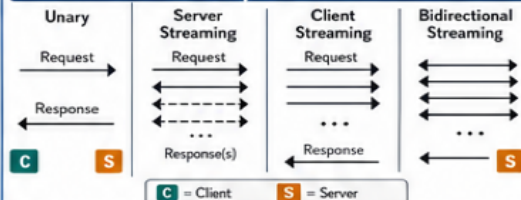
8 PRODUCTION

- Internal Microservices** high throughput, low latency
 - Polyglot Platforms** many languages, one contract
 - Mobile Backends** efficient over cellular
 - Real-time Streams** telemetry, gaming, IoT
 - ML Inference Services** streaming requests/responses
 - Batch/Chunked Uploads** client streaming
 - Control Planes** cluster/infra management
 - Edge & IoT Gateways** efficient, low overhead
 - Storage Systems** replication, metadata ops
 - B2B Integrations** partner APIs (with gateways)
- Build for internal systems first; adapt for external via gateways.

3 INTERNALS

- Protobuf field tags & wire types (varint, 64-bit, length-delimited, 32-bit, fixed64, fixed32, bool)
- Backward-compatible evolution: add fields, reserve numbers/names, never change type
- gRPC message = length-prefixed Protobuf (5B header: 1B compressed flag + 4B length)
- HTTP/2 streams: bidirectional, multiplexed over one TCP connection
- HPACK compresses headers; reduces overhead
- Trailers carry grpc-status, grpc-message, and custom metadata
- Connection/Channel reuse: pick_first, round_robin, least_request, xDS (envoy), outlier detection
- Interceptors / middleware (client & server) logging, auth, metrics, retries, tracing
- Deadlines & cancellation propagate via context grpc-timeout; client cancel → RST_STREAM
- Metadata: headers & trailers (auth, trace, custom) binary-safe key/value pairs

RPC Styles



6 DISCUSSION

- Deadlines:** always set; propagate budgets end-to-end
- Retries:** use with care; only on safe errors; respect budgets (UNAVAILABLE, DEADLINE_EXCEEDED)
- Retry safety:** prefer idempotent methods; use idempotency keys
- Status codes:** 16 canonical codes; map to semantics (OK, INVALID_ARGUMENT, NOT_FOUND, PERMISSION_DENIED, UNAVAILABLE, RESOURCE_EXHAUSTED, INTERNAL, ...)
- Rich errors:** google.rpc.Status + Any details for structured errors
- Backpressure & flow control:** HTTP/2 windows; pause reads when overwhelmed
- Streaming lifecycle:** onNext → half-close → complete or error; handle cancellations
- Keepalive:** detect dead peers; tune pings & timeouts
- Load balancing:** client-side LB via resolver & policy; xDS aware
- Connection pinning:** same channel per authority; avoid per-RPC
- Health checking:** gRPC health proto for liveness/readiness
- Reflection:** server reflection for tooling & dynamic clients
- Gateway / transcoding:** serve REST/JSON via gRPC-Gateway (HTTP/JSON → gRPC)
- Schema evolution:** additive changes; unknown fields ignored
- Unknown fields:** preserved by Protobuf; forward compatible

9 INTERVIEW THINKING

- Clarify: topology, clients, languages, sync vs async needs
- Define contract: domain model, .proto, versioning strategy
- Choose RPC type: unary vs (server/client/bidi) streaming
- Establish resilience: deadlines, retries, idempotency, budgets
- Design data flow: backpressure, flow control, chunking limits
- Security & governance: mTLS, authZ, metadata, quotas
- Quantify & observe: latency targets, throughput, p99, metrics, tracing, alerting, capacity plan

10 SENIOR NOTES

- Every RPC needs a deadline.
- Treat remote calls as failure-prone.
- Evolve schemas additively; never break consumers.
- Bound stream length & message sizes.
- Avoid retry amplification; use budgets & backoff + jitter.
- Reuse channels; limit connections per authority.
- Propagate trace context & metadata.
- Test failure modes: timeouts, cancel, overload, partial replies.

11 MENTAL HOOK



gRPC makes remote calls feel local; senior design remembers they still fail like networks.

TCP

Reliable, ordered byte streams across networks

1 WHY

TCP delivers a reliable, ordered byte stream between two processes over an unreliable IP network.

Rationale

- Lossy, duplicated, out-of-order networks need reliability & ordering.
- End-to-end semantics; apps get a stream, not packets.
- Built-in flow & congestion control protect hosts and the network.

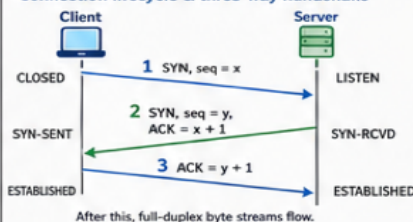
Mental model

Two reliable pipes with feedback. Sender fills the pipe; receiver acks what it has safely received.



2 CORE IDEA

Connection lifecycle & three-way handshake



Termination (common paths)

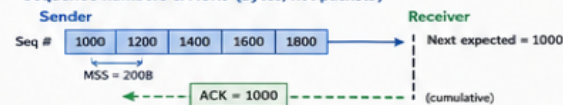
Active Closer	Passive Closer
FIN (send)	ACK
→ FIN-WAIT-1	→ CLOSE-WAIT (app closes)
← ACK	→ FIN-WAIT-1
→ FIN-WAIT-2	← ACK
← FIN	→ CLOSED
→ TIME-WAIT (2*MSL)	
← ACK	
→ CLOSED	

Key states

CLOSED
LISTEN
SYN-SENT
SYN-RCVD
ESTABLISHED
FIN-WAIT-1
FIN-WAIT-2
CLOSE-WAIT
LAST-ACK
TIME-WAIT

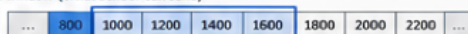
3 INTERNALS

Sequence numbers & ACKs (bytes, not packets)

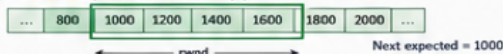


Sliding windows

Send window (what sender can send)



Receive windows (what receiver can accept)



Header (min 20B)

Src Port	Dst Port	Sequence Number
Acknowledgment Number		
Data Offset	Flags	Window Size
Checksum	Urgent Pointer	Options (MSS, SACK, TS)

MSS ≤ MTU - IP(20B) - TCP(20B) - options

Flags

SYN ACK FIN
RST PSH URG
ECE CWR

4 COMPLEXITY

Reliability mechanisms

- ACKs: cumulative; ACK = next byte expected.
- Retransmission Timeout (RTO):
 $RTO = SRTT + \max(G, K \cdot RTT_{VAR})$
(RFC 6298; typical $K=4$)
- Fast Retransmit: on 3 duplicate ACKs → retransmit missing segment.
- SACK (Selective ACK): receiver reports received blocks → efficient loss recovery.
- Checksums: 16-bit one's complement over header + data (stronger hash in practice with modern NIC offload).

Control loops

Flow Control (receiver → sender)	Congestion Control (network → sender)
• Uses rwnd (receiver window)	• Uses congestion window (cwnd)
• Protects receiver buffer	• Protects network
• Backpressure: rwnd=0 → sender stops, uses zero-window probes	• Adapts sending rate based on loss/latency signals

Effective send window = min(rwnd, cwnd)

5 TRADE-OFFS

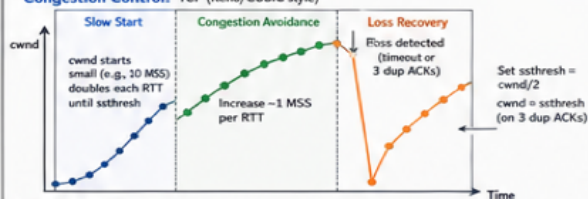
Goal	Benefit	Cost / Trade-off
Reliability	No loss to app, ordered delivery	Retransmissions add latency and bandwidth
Congestion Control	Fair sharing, network stability	Throughput limited by loss/RTT; ramp-up time
In-Order Delivery	Simple API, no reordering	Head-of-line blocking on loss
Flow Control	Protects receiver	Backpressure can reduce throughput
Large Buffers	Higher BDP utilization	More memory, higher latency
Small MSS	Better loss isolation, faster recovery	More headers, lower efficiency
Keepalive	Detect dead peers	Extra traffic (idle cost)

Key cost drivers



6 DISCUSSION

Congestion Control: TCP (Reno/CUBIC style)



Loss detection

- 3 Duplicate ACKs → Fast Retransmit + Fast Recovery (halve cwnd)
- Timeout → cwnd = 1 MSS, back to Slow Start

Flow vs Congestion

Signal	Action (Who)	What it limits	Example
rwnd full/zero	Reduce/stop (Sender)	Receiver buffer	App slow reader
Loss / ECN / High delay	Reduce cwnd (Sender)	Network capacity	Congestion

7 RELATED

Feature	TCP	UDP	QUIC (over UDP)
Connection	Connection-oriented	Connectionless	Connection-oriented
Reliability	Yes	No	Yes (per stream)
Ordering	Yes (per conn)	No	Yes (per stream)
Congestion Ctrl	Built-in	No	Built-in (new CC)
Handshake	3-way (1-RTT)	None	1-RTT (0-RTT resumption)
Multiplexing	No (HoL blocking)	N/A	Yes (streams)
Security	Optional (TLS)	Optional	Built-in (TLS 1.3)
Typical Uses	HTTP/1.1, DB, RPC, MQ, files	DNS, VoIP, gaming, streaming, IoT	HTTP/3, APIs, real-time web

When to use what?

- Need reliability & ordering → TCP
- Need low latency, tolerate loss → UDP
- Need reliability + multiplexing + modern web → QUIC

Head-of-line (HoL) blocking
Lost segment stalls all later data in TCP. QUIC fixes with per-stream independence.



8 PRODUCTION

Where TCP is used



Operational knobs & observability

Knob / Metric	Purpose
MSS / MTU	Avoid fragmentation; set MSS properly
Send/Receive buffers	Tune for BDP; reduce drops/backpressure
Keepalive	Detect dead peers (idle connections)
RTO / SRTT / RTTVAR	Latency estimation; influences timeouts
cwnd / ssthresh	Congestion behavior visibility
Retransmits / Duplicate ACKs	Signal loss & network issues
TIME_WAIT count	Port exhaustion risk (tune with care)
Bytes in flight (BIF)	Approx inflight = min(cwnd, rwnd)

Common failure modes

Packet Loss	High Latency	Bufferbloat	Port Exhaustion	Midbox / Firewall
Timeouts, retransmits, low throughput	Low utilization, large queues, higher RTO	High delay, queue buildup, spikes	Too many TIME_WAIT connections	Drops, resets, idle timeouts, blackholes

9 INTERVIEW THINKING

Step-by-step decision process

- Clarify requirements
Reliable? Ordered? Throughput/latency goals? Bidirectional?
- Choose protocol
Need reliability & ordering → TCP (or QUIC if web/streams)
Low latency tolerant → UDP
- Estimate capacity
 $BDP = \text{Bandwidth} \times \text{RTT}$. Ensure buffers & cwnd ≥ BDP.
- Connection strategy
Reuse connections; keepalive; multiplex (when possible).
- Failure handling
Retries with backoff, idempotency, timeouts, circuit breakers.
- Observability
Track latency, loss, retransmits, cwnd, errors, socket stats.

Sample quantitative questions

- Given BW=100 Mbps and RTT=50 ms, what buffer to fill pipe?
 $BDP = 100 \text{e6 bps} \times 0.05 \text{ s} = 6.25 \text{e6 bits} \approx 0.78 \text{ MB}$
→ Window/buffers should be ≥ ~1 MB.
- If MSS=1460B and RTT=40ms, slow start reaches ssthresh=64 MSS. How long to exit slow start? $\log_2(64/1)=6$ RTTs ≈ 240 ms.
- Why 3 duplicate ACKs? Heuristic: likely loss without waiting for timeout.

10 SENIOR NOTES

- Design for failure: assume loss, reordering, node restarts, network partitions, asymmetric paths.
- Tune with data: measure RTT, loss, cwnd, BIF, queue depth; change one variable at a time.
- Avoid cargo-cult tuning: big buffers without fixing loss/latency can hide problems (bufferbloat).
- Prefer higher-level protocols: HTTP/2/3, gRPC, QUIC for features like multiplexing, TLS, better congestion control.
- Capacity planning: size for peak BDP, connection churn, and TIME_WAIT impact.
- Security & correctness: validate inputs, handle half-closes, timeouts, and retries idempotently.

11 MENTAL HOOK

TCP is a carefully balanced dance between optimism and caution:
send more to be **fast**,
but back off to be **fair**.

- Optimism (Speed)
Slow start, bigger windows, keep the pipe full.
- Caution (Stability)
Detect congestion/loss, reduce rate, protect the network.
- Reliability (Correctness)
ACKs, retransmissions, checksums ensure delivery.
- Backpressure (Kindness)
Flow control pushes back so receivers aren't overwhelmed.
- Simplicity at the API
Apps read/write a stream; TCP handles the mess.

UDP

Minimal datagrams with application-owned reliability

1 WHY

- ⚡ **Connectionless, stateless**
no handshake, no session
- 🕒 **Low latency & low overhead**
minimal header (8 bytes)
- 📏 **Message boundaries**
each datagram preserved
- 🔧 **Flexible & app-controlled**
reliability, ordering, flow, rate
- 👥 **Scales & simple**
easy to implement & fan-out
- 📡 **Works with multicast/broadcast**
1 → many distribution

2 CORE IDEA



Guarantees

- ✓ Message boundaries
- ✓ Checksum (error detect)
- ✗ No delivery guarantee
- ✗ No ordering guarantee
- ✗ No flow/congestion control
- ✗ No retransmission

3 INTERNALS

UDP Header (8 bytes)

0	15	16	31
Source Port (16)		Destination Port (16)	
Length (16)		Checksum (16)	
Data (0 .. 65,507 bytes)			

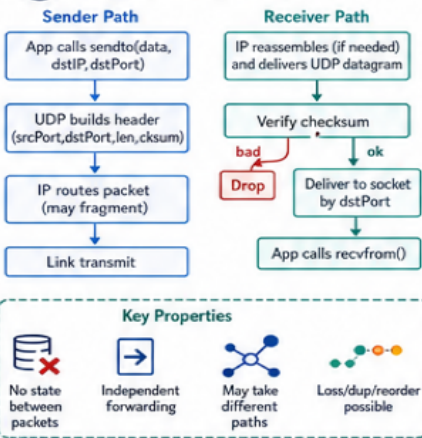
- **Ports:** identify sending/receiving applications
- **Length:** header + data (min 8, max 65,535)
- **Checksum:** 1's complement over pseudo-header + UDP header + data (IPv4 optional, IPv6 mandatory)

Pseudo-header (used by checksum)

IPv4	IPv6
• Src IP (32)	• Src IP (128)
• Dst IP (32)	• Dst IP (128)
• Zero (8)	• UDP Length (32)
• Protocol (8) = 17	• Zero (24)
• UDP Length (16)	• Next Header (8) = 17

Error detection only. Corrupted datagrams are dropped.

4 COMPLEXITY



5 TRADE-OFFS

Dimension	UDP (Pros)	UDP (Cons)
⌚ Latency	Low overhead no handshake	You build retries if needed
⚖️ Overhead	Header = 8 bytes	Less features in-network
🛡️ Reliability	App can tailor what/when/how	No built-in delivery or ordering
🔧 Control	Full control of rate & behavior	Easy to misuse; causes loss
📡 Distribution	Multicast & broadcast	TCP not suitable for fan-out
🔒 NAT/Firewall	Works with connectionless NAT mappings	Mapping timeout can drop flows
🔍 Debugging	Visible, simple behavior	App must add observability

6 DISCUSSION

What UDP does

- ✓ Delivers independent datagrams end-to-end
- ✓ Preserves message boundaries
- ✓ Provides ports & checksum (error detection)

What UDP does NOT do

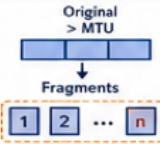
- ✗ No connection or handshake
- ✗ No reliability (delivery/ack/retry)
- ✗ No ordering
- ✗ No flow control
- ✗ No congestion control

Failure Modes

- 🗑️ Loss (dropped)
- 📄 Duplication
- 🔄 Reordering
- 🔥 Corruption (detected & dropped)

MTU, Fragmentation & Reassembly

- IP MTU limits payload (e.g., Ethernet ~1500 bytes)
- If packet > MTU and DF=0 → IP fragments
- Lost fragment → whole datagram lost
- Best practice: keep UDP payload ≤ Path MTU (or use PMTUD / PLPMTUD)
- IPv6 routers do NOT fragment; sender must fit MTU



7 RELATED

	TCP	QUIC
Connection	Connection-oriented	Connection-oriented over UDP
Reliability	Reliable, in-order	Reliable, multiplexed streams
Handshake	3-way handshake (1-RTT)	0-RTT / 1-RTT (crypto handshake)
Congestion Control	Built-in (AIMD)	Built-in (QUIC CC)
Overhead	Higher	Lower (UDP + TLS 1.3)
Head-of-line Blocking	Yes (per-connection)	No (per-stream)
Use When	Strong reliability, ordered bytes	Low-latency web, mobile, multiplexing

Other related protocols: SCTP (message-oriented, multi-homing), DCCP (congestion-aware), RUDP variants (app-level reliability).

8 PRODUCTION

DNS
Queries/responses (UDP/53)

Media Streaming
RTP/RTCP, Live video, VoIP

Online Gaming
State updates, low latency

Telemetry/Logs
Metrics, traces, event streams

Service Discovery
mDNS, SSDP, NTP, Syslog

TFTP, DHCP, SNMP, CoAP, STUN, QUIC

Operational Concerns

- Choose ports (well-known vs ephemeral)
- Handle NAT timeouts (keepalive if needed)
- Add app-level:
 - sequence numbers
 - ACKs / NACKs
 - retries with backoff
 - deduplication
 - idempotency keys
- Rate limit & budget loss
- Observability: loss, latency, reorder, dup

Golden Rule: Make best-effort work for your SLA.

9 INTERVIEW THINKING

- 1 Clarify requirements → Latency? Ordering? Reliability? Throughput? Fan-out?
- 2 Choose transport → UDP if latency or broadcast matters & you can handle reliability.
- 3 Design message format → Version, type, length, seq, ts, checksum (app if needed)
- 4 Reliability strategy → ACK/NACK? Retry? FEC? Duplicate suppression? Idempotency?
- 5 Flow & rate control → Token bucket / leaky bucket / adaptive to loss/RTT
- 6 Failure handling → Timeouts, retries, circuit breaker, backoff, degradation paths
- 7 Observability & ops → Metrics: loss, RTT, reorder, dup, P50/P99, drop reasons

Think in trade-offs. Make the simplest thing that meets SLA.

10 SENIOR NOTES

- ✓ **Simplicity scales.** Fewer states, easier to operate.
- ⌚ **Correctness over features.** Define semantics, then add reliability.
- 🔧 **Own the edges.** Handle NAT, MTU, retries, backoff.
- 🔧 **Measure & adapt.** Instrument, detect issues, respond.
- 🔑 **Idempotency is power.** Safe retries, de-dup, at-least-once.
- 📈 **Progressive enhancement.** Start minimal, iterate with data.
- 🔒 **Security matters.** Validate inputs, rate limit, auth, use DTLS if needed.

11 MENTAL HOOK

“UDP is like sending postcards.”



Great for **speed**. Your app provides the **guarantees**.

DNS

Distributed hierarchical name resolution

1 WHY



Maps human-friendly names to IPs and other data.



Scales globally via a distributed hierarchy.



Decouples names from locations → agility.

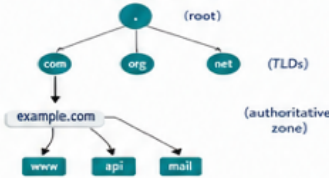


Enables many services: email, web, APIs, CDNs, service discovery.

2 CORE IDEA

- ✓ The DNS namespace is a tree (the "DNS Global Namespace").
- ✓ Authority is delegated down the tree (zones) to different organizations.
- ✓ Resolvers resolve names by asking other servers until an authoritative answer is found, then cache it.

HIERARCHY MENTAL MODEL

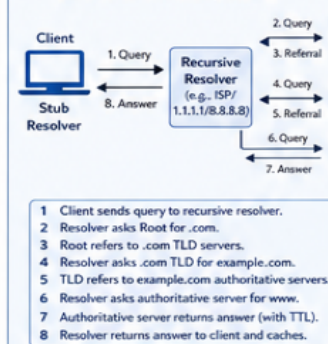


Zones = portions of the namespace for which a set of authoritative name servers is responsible.

3 INTERNALS

LOOKUP FLOWS

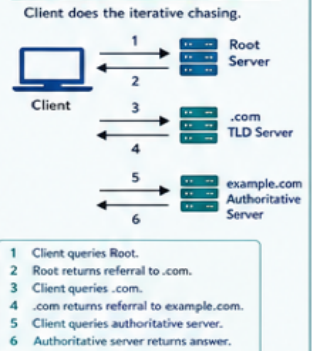
A. RECURSIVE LOOKUP (typical)



STUB RESOLVER (on device/OS)

- Does minimal DNS logic.
- Forwards queries to recursive resolver via UDP/TCP (or DoH/DoT).
- Caches per OS rules (respecting TTLs).

B. ITERATIVE LOOKUP (custom)



TRANSPORT

- UDP 53** • Default
- Low latency
- No connection
- 512B typical limit
- TCP 53** • For large responses
- Zone transfers (AXFR)
- DNSSEC (big answers)
- Connection oriented

4 CORE DATA & RECORD TYPES

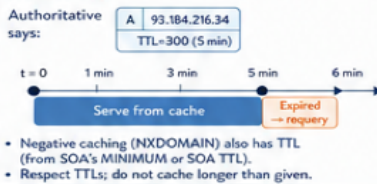
Type	Meaning	Examples / Notes
A	IPv4 address	example.com. A 93.184.216.34
AAAA	IPv6 address	example.com. AAAA 2606:280c::1
CNAME	Canonical name	www → example.com
MX	Mail exchanger	example.com MX 10 mail.example.com
NS	Name server	example.com NS ns1.example.com
SOA	Zone start of authority	Zone metadata, serial, timers
TXT	Text data	SPF, verification, arbitrary text
PTR	Pointer (reverse)	IP → name (in-addr.arpa / ip6.arpa)
SRV	Service locator	_sip._tcp.example.com
CAA	Cert. authority auth	Which CAs can issue certs
DNSKEY	DNSSEC keys	Public keys for the zone
DS	Delegation signer	Hash of DNSKEY (in parent zone)

Other types: HINFO, LOC, TLSA, NAPTR, SVCB/HTTPS, etc.

5 CACHING & TTL



TTL EXAMPLE



- Negative caching (NXDOMAIN) also has TTL (from SOA's MINIMUM or SOA TTL).
- Respect TTLs; do not cache longer than given.

6 COMPLEXITY

TIME / LATENCY	• Best case (warm cache): ~1 RTT (resolver)
	• Cold, cached at resolver: 1-2 RTT
	• Full iterative chain (worst): 4-6 RTTs (Root → TLD → Auth)
COST	• Resolver CPU, network RTTs, bandwidth
	• More queries = more cost
	• Anycast & caching reduce global cost
SPACE	• Resolver caches many records
	• Cache size vs hit ratio
	• Zone data size at authoritative servers

Rough sizes: DNS message ~60B question, ~100-400B typical answer, up to 64KB over TCP.

7 TRADE-OFFS

Strengths	Limitations
✓ Highly scalable & distributed	⚠ Eventual consistency (propagation delay)
✓ Robust to failures (redundant)	⚠ Complex to operate securely
✓ Low-latency lookups (with cache)	⚠ DDoS target (amplification)
✓ Hierarchical delegation	⚠ Cache poisoning risks
✓ Extensible (many record types)	⚠ Privacy leakage (by default)
✓ Works across networks	⚠ Hard to debug at scale

Balance: freshness vs performance vs cost vs security.

8 DISCUSSION (FAILURES & REALITIES)

COMMON FAILURE MODES	PROPAGATION & CONSISTENCY
⚠ NXDOMAIN: Name does not exist.	Change at Authoritative
⚠ Timeout: No response from server.	Parent (NS/DS) cached
⚠ SERVFAIL: Server failure.	Resolver caches
⚠ REFUSED: Query refused (policy).	TLD cache
⚠ Truncation (TC=1): UDP response too big.	Global resolvers
⚠ Cache poisoning: Bogus data injected.	Minutes Hours Days
⚠ DDoS: Overwhelmed by traffic.	

RULE OF THUMB: TTL × (1-2)

- Lower TTL = faster propagation, higher load
- Higher TTL = stability, slower changes

9 RELATED (ALTERNATIVES & COMPLEMENTS)

ALTERNATIVES / COMPLEMENTS	SECURITY & PRIVACY
• Hosts file / local DNS	• DNSSEC: Authenticates data (integrity & origin)
• Static mappings (small scale)	• DoH (HTTPS / 443): Encrypts DNS in HTTPS
• Service discovery (internal): Consul, etcd, ZK, Kubernetes DNS	• DoT (TLS / 853): Encrypts DNS in TLS
• CDN / Anycast: Deliver content closer to users	• QNAME minimization: Less info leaked upstream
• GeoDNS / Traffic steering: Route based on location/policy	• EDNS(0) + Buffer size: Fewer truncations
• Load balancers: L4/L7 for traffic distribution	

10 PRODUCTION CHECKLIST

USEFUL COMMANDS
✓ Authoritative servers on anycast, redundant networks.
✓ Short but sane TTLs (e.g., 60-300s common).
✓ Health checks & monitoring (latency, errors, RRL drops, cache hit ratio).
✓ DNSSEC enabled end-to-end.
✓ Protect against DDoS (rate limiting, RRL, WAF/Anycast).
✓ Automate zone changes (CI/CD), version control, safe rollouts.
✓ Test propagation before/after changes.
✓ Have runbooks for incidents.

11 INTERVIEW THINKING

1. Clarify	SAMPLE PROMPTS YOU CAN STEER WITH
What name? From where? Pattern (web, email, internal)?	• "Walk me through resolving www.example.com."
2. Mental model	• "Why are there TLD servers?"
Hierarchy, delegation, caching, authoritative vs recursive.	• "What's the difference between recursive and iterative?"
3. Path	• "How does caching help?"
Stub → Resolver → Root → TLD → Authoritative → Answer.	• "How would DNSSEC work here?"
4. Analyze	• "What happens if the authoritative server is down?"
Latency? Error cases? TTL? Consistency needs?	• "How would you reduce propagation time?"
5. Choices	
Record types, TTLs, DNSSEC, DoH/DoT, anycast, monitoring.	
6. Trade-offs & risks	
Cost vs latency vs freshness vs security.	

SENIOR NOTES

- ★ Design for failure: multi-region anycast, no single points.
- ★ Pick TTLs based on change frequency & risk.
- ★ Minimize dependencies between zones.
- ★ Use DS at parent to enable DNSSEC; automate key rollover.
- ★ Watch for amplification: disable unnecessary recursion; use RRL.
- ★ Prefer small responses (use CNAME, not big TXT blobs).
- ★ Measure: cache hit ratio, p95 latency, SERVFAIL rate.

11 MENTAL HOOK



LOAD BALANCING

Distributing traffic across healthy capacity

1 WHY

- SCALE**
Spread load to scale horizontally.
- HIGH AVAILABILITY**
Route around failures; maximize uptime.
- PERFORMANCE**
Lower latency (p50/p99); maximize throughput.
- EFFICIENCY**
Better utilization; reduce hotspots.

SOLVES

- ✓ Traffic spikes
- ✓ Server failures
- ✓ Zone/AZ outages
- ✓ Maintenance
- ✓ Uneven load (skew)
- ✓ Overload protection

Goal: Send each request to the best available instance—*fast, fair, and safe*.

2 CORE IDEA

Act as the traffic cop between clients and a pool of backends.

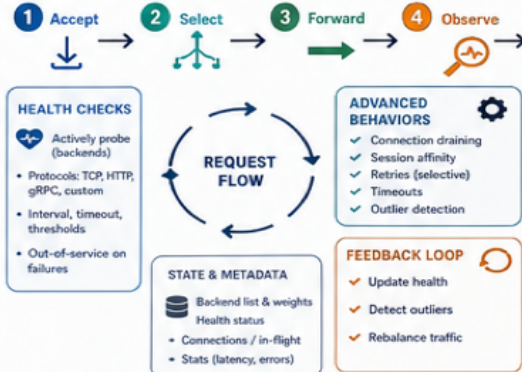


Continuously check health, understand load, and route to the best healthy target.

HAPPENS AT TWO LEVELS

- L4 (Transport)**
TCP/UDP, IP:Port
Fast, simple, opaque
- L7 (Application)**
HTTP/HTTPS, Host, Path, Header
Smarter routing, more features

3 INTERNALS



4 COMPLEXITY

Time Complexity (per request)

ALGORITHM	TIME
Round Robin (w/o weights)	O(1)
Weighted Round Robin	O(1)
Least Connections	O(1) (approx)
Random (uniform)	O(1)
Consistent Hashing	O(1)
Power of Two Choices	O(1)
Latency Aware (EWMA)	O(1)

Space Complexity (control plane)

- O(N) Store backend list & health (N = backends)
 - O(N) Per-backend stats (latency, errors, load)
 - O(1) Per-request routing state
- N = number of backends

5 TRADE-OFFS

DIMENSION	STRENGTHS	LIMITATIONS
Latency	L4 is fastest	L7 adds parse cost; more features
Throughput	Distributes load; higher utilization	Overhead at LB; encryption cost
Fairness	Algorithms & weights improve balance	Skewed keys, heavy hitters still happen
Resilience	Health checks, retries, outlier detection	False positives can eject good hosts
Stickiness	Affinity improves cache locality	Can cause imbalance and hotspots
Operational Cost	Scales well; automation	Complex configs; more moving parts
Visibility	Metrics, traces, percentiles	Needs good instrumentation

6 DISCUSSION

ALGORITHMS (L4/L7)	KEY FEATURES
• Round Robin (RR) • Weighted RR (WRR) • Least Connections (LC) • Weighted Least Conns (WLC) • Random (uniform) • Power of Two Choices (P2C) • Consistent Hashing (ring) • Latency Aware (EWMA)	✓ Health checks ✓ Connection draining ✓ Session affinity (cookie/IP) ✓ Retries (idempotent only) ✓ Timeouts (connect & request) ✓ Outlier detection (eject) ✓ Rate limiting / Overload protect
ROUTING EXAMPLES (L7)	METRICS THAT MATTER
- Host: api.example.com - Path: /v2/* → v2-svc - Header: X-Region: eu - Method: POST → write-svc	• RPS, Throughput • Error rate (Sxx) • Latency: p50 / p95 / p99 • In-flight / Conns • Healthy backends • Retry rate, Timeout rate • Rebalance rate, Ejections

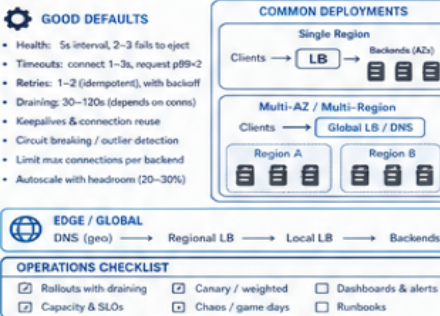
7 RELATED

- DNS LOAD BALANCING**
Global/basic, low cost, coarse control.
- SERVICE DISCOVERY**
Find backends (Consul, etcd, ZK).
- SERVICE MESH**
Envoy sidecars, mTLS, advanced routing.
- API GATEWAY**
Auth, rate limit, routing, aggregation.
- CDN / EDGE ROUTING**
Global caching, geo routing.
- ANYCAST**
Route to nearest edge via network.

L4 vs L7

- L4**
 - TCP/UDP
 - IP:Port
 - Fast
 - No payload visibility
 - Use for raw perf, TLS pass-through
- L7**
 - HTTP/HTTPS
 - Host/Path/Header
 - Rich routing
 - WAF, auth, rewrite
 - Use for app-aware features

8 PRODUCTION



9 INTERVIEW THINKING

- CLARIFY**
Scope: L4 or L7? Geo? Traffic profile? Stickiness? Failure model? SLOs?
 - REQUIREMENTS**
RPS, latency (p50), availability, consistency, traffic patterns, growth.
 - DESIGN CHOICES**
Algorithms, health, timeouts/retries, affinity, routing rules, global vs local.
 - FAILURE MODES**
Backend down, AZ down, LB down, skew, overload, noisy neighbor.
 - TRADE-OFFS**
Latency vs features, fairness vs stickiness, cost vs control.
 - SCALE & OPERATE**
Autoscale, metrics, alerts, runbooks, rollouts & draining.
- Think in flows, states, and failure—then in trade-offs.

10 SENIOR NOTES

- SKREW IS INEVITABLE**
Heavy hitters, uneven keys, diurnal patterns. Use sharding keys, quotas, P2C, autoscale.
- OVERLOAD KILLS P99**
Protect the tail: timeouts, limits, backpressure, shed load early, queue carefully.
- MEASURE WHAT MATTERS**
p99, error budget, saturation (utilization), and cost per request.
- BE SLO-DRIVEN**
Design to hit SLOs, not just averages. Budget latency across hops.

- FAIL OPEN vs FAIL CLOSED**
Know when to serve partial vs reject. Prefer graceful degradation.
- OUTLIERS LIE**
Tune ejection: consecutive failures, successful probes to heal.
- GLOBAL IS HARD**
Latency variance, data residency, and regional failover plans.
- SIMPLICITY WINS**
Prefer simple algorithms + good observability over clever magic.

11 MENTAL HOOK



WEBSOCKETS

Full-duplex messages over persistent connections

1 WHY



LOW LATENCY

No repeated HTTP handshakes.
Messages flow both ways instantly.



FULL-DUPLEX

Client and server can send independently.

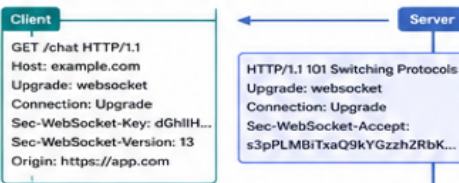


REAL-TIME USE CASES

Chat, presence, live dashboards,
collab, multiplayer, IoT.

3 INTERNALS

A. HTTP UPGRADE HANDSHAKE



B. DATA FLOW AFTER UPGRADE

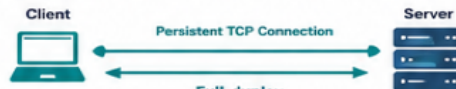


C. CONNECTION TEARDOWN

Either side can close.



2 CORE IDEA



One connection. Many messages.
No headers per message.

KEY PROPERTIES

- ✓ Stateful connection over TCP (port 443 w/ TLS).
- ✓ Message-oriented via frames.
- ✓ Ordered & reliable delivery (TCP).
- ✓ Application-level heartbeats & flow control.

4 COMPLEXITY

Aspect	What to Know
Time	Handshakes are O(1). Per-message cost: lightweight framing. Lower latency vs HTTP polling.
Space	Connection state per client. Memory for buffers & queues. Careful with large fan-out.
Network	One TCP/TLS connection. Minimal framing overhead (2-14 bytes + payload).
Ordering	Guaranteed in-order delivery per connection (TCP).
Backpressure	Must manage send buffers, pauses, and slow consumers.
Complexity	Higher than HTTP: stateful, timeouts, reconnects, scaling.
Costs	Fewer requests = lower CPU, bandwidth & latency.

6 DISCUSSION

- When would you choose WebSockets? When not?
- How do you handle disconnects & reconnects?
- How do you ensure ordering & avoid duplicates?
- How do you handle backpressure and slow clients?
- How do you scale to millions of connections?
- How do you secure and authenticate connections?
- How would you support presence / typing indicators?
- What metrics & alerts would you track?

8 PRODUCTION

- CONNECT**
Use wss:// over TLS. Send auth token in query or headers during upgrade (or post-auth message).
- HEARTBEAT**
Ping/pong or app-level heartbeat.
Detect dead peers & network drops.
- BACKPRESSURE**
Bounded send queues. Pause reads or drop.
Apply per-connection limits.
- RECONNECT**
Exponential backoff with jitter.
Resume state & re-subscribe.
- SCALING**
Terminate WS at gateways (L7). Route via consistent hash. Publish/subscribe (Redis, NATS, Kafka) to fan-out across nodes.
- OBSERVABILITY**
Track: conns, msgs/sec, p95 latency, queue depth, errors, reconnect rate.
- SECURITY**
TLS, origin checks, authn/z, rate limits, message validation.

9 INTERVIEW THINKING

- CLARIFY REQUIREMENTS**
Two-way or one-way? Real-time?
Traffic pattern? Message size?
- EVALUATE OPTIONS**
WebSocket vs SSE vs Polling.
Check constraints (firewalls, infra).
- CHECK TRADE-OFFS**
Latency, cost, complexity,
ordering, reliability needs.
- DESIGN AT A HIGH LEVEL**
Connection flow, scaling, backpressure,
reconnect, observability.
- PLAN FOR FAILURE**
Disconnects, retries, idempotency,
duplicates, monitoring.
- MAKE A CALL**
Pick the simplest solution that
meets SLOs and constraints.

10 SENIOR NOTES

- ★ Design for the real world, not the happy path.
Every connection will drop—assume it.
- ★ Limit blast radius.
Shard by tenant/user, per-connection limits,
timeouts, and circuit breakers.
- ★ Observe everything.
Connection churn, msg/sec, p95 send latency,
queue depth, errors.
- ★ Backpressure over blind buffering.
Prefer shedding or degrading to OOM.
- ★ Idempotent & replay-safe messages.
Clients can reconnect and miss or duplicate.
- ★ Prefer simplicity.
If one-way or low-frequency, SSE is often better.
- ★ Align with product & infra constraints.
E.g., corporate proxies may block WebSockets.

11 MENTAL HOOK

WEBSOCKETS ARE A "LIVE PHONE CALL"



One line. Both sides talk. No dial-up.

5 TRADE-OFFS

STRENGTHS

- ✓ Low latency, real-time.
- ✓ Full-duplex.
- ✓ Efficient (no per-message HTTP).
- ✓ Leverages existing ports (80/443).
- ✓ Works with TLS for security.

LIMITATIONS

- ✗ Stateful: harder to scale.
- ✗ Proxies/firewalls may block.
- ✗ More complex to implement.
- ✗ Not ideal for large binary streams (use QUIC/gRPC).
- ✗ Mobile networks are flaky.

RULE OF THUMB

Use WebSockets for real-time, two-way, low-latency.
Use simpler options when you don't need all that.

7 RELATED

Feature	WebSockets	SSE (Server-Sent Events)	Polling (Short / Long)
Direction	Bi-directional	Server → Client	Bi-directional (via requests)
Transport	WS over TCP	HTTP over TCP	HTTP over TCP
Latency	Very low	Low	High (interval)
Overhead	Low	Low	High
Proxies	Sometimes blocked	Usually allowed	Always allowed
Use when	Real-time, two-way	Server push only	Fallback / simple
Browser API	WebSocket	EventSource	fetch / XHR

Good default: Start with SSE if one-way. Use WebSockets when you need two-way and very low latency.

FAILURE MODES & HOW TO HANDLE

Network drop / mobile handoff	Ping/pong, short timeouts, reconnect with backoff, resume state.
Server overload	Backpressure, shed load, limit conns, queue caps, autoscale.
Slow consumer	Per-conn send queue limits, drop or disconnect.
Proxy / idle timeout	Keepalives below idle timeout, document requirements.
Duplicate messages	Idempotent messages, unique IDs, de-dup on client.
Out-of-order across topics	Per-connection order only; add sequence numbers if needed.

CDN

Serving content from the network edge

1 WHY



Lower latency
Serve from close to users
(ms instead of 100s ms)



Higher performance
Offload origin, absorb spikes,
higher throughput



More reliable
Redundancy across PoPs,
DDoS absorption

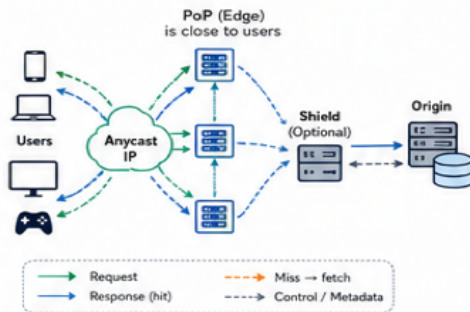


Lower egress cost
Cache at edge, reduce
origin bandwidth



Better experience
Consistent, fast, global delivery
for web, video, APIs, downloads

2 CORE IDEA



Mental model

User → Anycast → PoP → Cache? → (Hit)
↳ No → Shield (opt.) → Origin → Store → Respond

3 INTERNALS

Request flow (detailed)

- 1 DNS resolves to anycast IP (nearest PoP)
- 2 TCP connect (and TLS handshake)
- 3 PoP receives request
- 4 Compute cache key
- 5 Lookup in cache
 - HIT → serve, update stats/TTL
 - MISS → go to 6
 - STALE → serve stale, go to 6 (async)
- 6 (Optional) Shield PoP lookup
- 7 Fetch from Origin
- 8 Store in cache, respond to user

Cache key composition

Scheme Host Path Query* Headers* ...

*Only whitelisted headers (e.g., Accept-Encoding, Cookie for whitelisted paths, Authorization for private content)

Object states



Revalidation (validators)

- Conditional request with validators
- If-None-Match (ETag) / If-Modified-Since (Last-Modified)
- 304 Not Modified → update metadata, keep body

4 COMPLEXITY



Latency

- DNS anycast: ~10–50 ms
- TLS handshake: 1–2 RTT
- Edge hit: 10–40 ms typical
- Miss to origin: +RTT to origin



Space

- Cache size at each PoP (GB–TB)
- Eviction: LRU / LFU / Size-based
- Compression: gzip/br/brotli
- Objects vary: KB (HTML) → GB (video)



Cost

- CDN request + bandwidth fees
- Origin offload reduces egress cost
- More cache HIT % → lower cost
- Features (WAF, TLS, logs) add cost

Key metric: **HIT RATIO** = Hits / (Hits + Misses)
Higher hit ratio → lower latency & cost

5 TRADE-OFFS

Dimension	Optimize for	Trade-off
Latency	Edge HIT, more caching	Staleness risk, invalidation complexity
Freshness	Short TTL, strict revalidation	Higher origin load, lower hit ratio
Cost	Longer TTL, large cache	More storage cost, stale content risk
Security	Open caching (wider keys)	Poisoning risk, leak private data
Features	WAF, TLS, logging, transform	Higher cost, extra latency
Simplicity	Simple rules, broad caching	Less control, edge cases

6 DISCUSSION

Strengths

- ✓ Massive latency reduction with global PoPs
- ✓ Scales to billions of requests
- ✓ Protects origin, absorbs traffic spikes/DDoS
- ✓ Rich edge features (WAF, routing, transforms)
- ✓ Lower bandwidth cost with high hit ratio

Limitations / Risks

- ⚠ Stale content if TTL/purge not managed
- ⚠ Cache poisoning / key manipulation risk
- ⚠ Complex invalidation for dynamic content
- ⚠ Vendor lock-in & cost unpredictability
- ⚠ Not a silver bullet for personalized data

When not to use

- ✗ Highly dynamic, per-user content (no cache benefit)
- ✗ Strict real-time freshness (sub-second)
- ✗ Very small audience / no global users

7 RELATED



Reverse Proxy
Terminates client conn., forwards to origin
(Nginx, Envoy) — no global distribution



Load Balancer
Distributes traffic across origin servers
(ALB, NLB) — within a region



Object Storage + Gateway
Store (S3/GCS) + CDN in front for delivery



Edge Compute
Run code at edge (KV, auth, AB test,
image resize, redirects)



Service Mesh / API Gateway
For service-to-service or API concerns,
not global content delivery

8 PRODUCTION

Architecture

- Multi-region origins
- Shield PoP in each region
- Global anycast network

Caching

- Set TTLs by content type
- Use validators (ETag/LM)
- Cache key: keep minimal

Invalidation

- Purge by URL / path / tag
- Versioned assets (fingerprints)
- Soft purge + revalidation

Security

- TLS everywhere (TLS1.2/1.3)
- WAF rules (OWASP Top 10)
- Prevent cache poisoning

Observability

- Metrics: hit ratio, latency, 4xx/5xx
- Logs & real-time dashboards
- Alerts on anomalies

Common status codes

200 OK	Served from origin
200 (From Cache)	Cache HIT (fresh)
304 Not Modified	Revalidated, not modified
404 Not Found	Not found (cached per TTL)
412 Precondition Failed	Validator mismatch
5xx (Origin)	Origin error
503 Service Unavailable	Rate limited / Overloaded

9 INTERVIEW THINKING

1. Clarify requirements

What content? Global? Latency SLO? Freshness? Security? Traffic pattern? Write vs read ratio?

2. Is CDN needed?

Can we cache it? Will it improve latency/cost/reliability?

3. What to cache?

Static assets, videos, images, docs, APIs (GET)?

4. Cache strategy

TTL? Validators? Cache key? Compression? Range requests?

5. Invalidation plan

How to purge/update? Versioning? Tags? Soft purge?

6. Risks & mitigations

Stale content, poisoning, DDoS, cost → how to handle?

Decision = Right content + Right TTL + Right key + Right invalidation + Security + Cost visibility

10 SENIOR NOTES

- ⚖ **Make trade-offs explicit:** Latency vs Freshness vs Cost.
- 🚫 **Design for failure:** PoP down? Origin down? Partial outages?
- 📊 **Use real metrics:** Hit ratio, p95 latency, origin RPS, bytes saved.
- 🔑 **Control the key:** Keep it minimal & stable; never cache by unbounded headers/cookies.
- 🛡 **Poisoning is real:** Validate inputs, normalize, use allowlists, use separate cache for authenticated content.
- ⚡ **Automate invalidation:** Prefer versioned assets + short TTL for HTML, purge APIs for emergencies.
- 💰 **Cost governance:** Monitor egress, requests, features; set budgets & alerts.
- 🌐 **Test from the edge:** Synthetic tests from multiple regions.

11 MENTAL HOOK

CDN is a Global Library with Smart Librarians



PoPs are everywhere.

Cache smart.

Keep users close, keep it fast.